

```

$ gcc -S while_loop_without_volatile.c
$ ls -l while_loop_without_volatile.s
-rw-rw-r-- 1 satya satya 482 Oct 24 12:27 while_loop_without_volatile.s
$ cat while_loop_without_volatile.s

```

Figure 1: Size of assembly code generated by the compiler without qualifying the variable as volatile

```

$ gcc -S while_loop_with_volatile.c
$ ls -l while_loop_with_volatile.s
-rw-rw-r-- 1 satya satya 501 Oct 24 14:20 while_loop_with_volatile.s
$ cat while_loop_with_volatile.s

```

Figure 2: Size of assembly code generated by the compiler with the variable qualified as volatile

that this piece of code is not achieving any valuable results; hence, the code will be optimised by removing this.

If one observes the code that follows below, the condition inside the *while* loop is replaced by the compiler to *while (TRUE)*. This is primarily done in compilers in the embedded systems environment, where generating optimal machine code is very important. As a programmer, if you are not aware of this, it could result in unexpected behaviour at runtime.

```

1 int main()
2 {
3     int i = 0;
4     int flag = 0;
5
6     //Some code
7
8     while(TRUE)
9     {
10         //Infinite loop
11     };
12 }
13
14 //rest of the code
15 return 0;
16 }

```

Now, the question is, “How can one confirm that the compiler is really optimising the code?” Let us check the size of the assembly code (call this *while_without_volatile.s*) generated by the compiler using the steps given below; the size is shown in Figure 1.

How to generate assembly code from the C source code in the gcc compiler

Step 1: Compile the C source code with the *-save-temps* option as shown below:

```
gcc -o sample sample.c -save-temps
```

Step 2: Check out the *sample.s* file generated by issuing the *ls* command.

When we compile code with the *-save-temps* option of

gcc, it generates three output files:

- Pre-processed code (with the *.i* extension)
- Assembly code (with the *.s* extension)
- Object code (with the *.o* option)

Now, if you observe Figure 1, the size is found to be 482 bytes in the fifth column. Next, qualify the flag variable to ‘volatile’ for the code shown in Illustration 1, and generate the assembly code (call this *while_with_volatile.s*) before checking the size by issuing the *ls* command. The size obtained in my system is shown in Figure 2.

Now, if you observe Figure 2, the size is found to be 501 bytes in the fifth column. So, when we compare the sizes of both the codes, with and without the ‘volatile’ key word, it is obvious that the compiler is not optimising the ‘flag’ variable when it is qualified as ‘volatile’.

Let us experiment further to explore where exactly the compiler is optimising the code. To find this out, apply the *vimdiff* command to the assembly codes generated with and without the keyword ‘volatile’—the difference is shown in Figure 3.

How to apply the vimdiff command

First, type:

```
vimdiff while_without_volatile.s while_with_volatile.s
```

In the disassembly of the non-volatile version (*while_loop_without_volatile.s*) of the *while* loop shown in Figure 3, the statements in lines 14 and 15 load the value of the flag into memory locations *[-8(%ebp) & -4(%ebp)]* outside the loop labelled *.L2*. This is because, since the flag variable is not declared volatile, the compiler assumes that its value cannot be modified outside the program. Having already read the value of the flag into memory locations *[-8(%ebp) & -4(%ebp)]*, the compiler omits reloading the value of the flag variable when optimisation is enabled, because its value cannot change. The result is ultimately the control getting into

Figure 3: Difference between the assembly codes generated with and without the ‘volatile’ key word